

Gated Delta Networks 论文精读

Improving Mamba2 with Delta Rule

Songlin Yang (MIT CSAIL), Jan Kautz (NVIDIA), Ali Hatamizadeh (NVIDIA)
ICLR 2025 · arXiv:2412.06464v3

全文逐段翻译与深度解读

Contents

1 摘要 (Abstract)	2
2 第1节: 引言 (Introduction)	2
3 第2节: 预备知识 (Preliminary)	4
3.1 2.1 Mamba2: 带衰减的线性注意力	4
3.2 2.2 Delta Networks: 带 Delta 规则的线性注意力	6
4 第3节: Gated Delta Networks	9
4.1 3.1 公式化: 门控 Delta 规则	9
4.2 3.2 案例研究: 大海捞针 (S-NIAH)	10
4.3 3.3 算法: 硬件高效的分块训练	11
4.4 3.4 Gated Delta Networks 和混合模型	12
5 第4节: 实验 (Experiments)	13
5.1 常识推理	14
5.2 真实世界上下文检索	15
5.3 长度外推	15
5.4 长上下文理解	16
5.5 训练吞吐量对比	16
6 第5节: 相关工作 (Related Work)	17
7 第6节: 结论 (Conclusion)	18
A 附录 A: 门控 Delta 规则的扩展 WY 表示证明	18
B 附录 B: 消融实验	19
C 附录 C: 参考文献一览表	21

1 摘要 (Abstract)

原文翻译

线性 Transformer 作为标准 Transformer 的高效替代方案已受到广泛关注，但其在检索和长上下文任务中的表现一直有限。为解决这些局限性，近期研究探索了两种不同的机制：用于自适应内存控制的**门控** (gating) 以及用于精确内存修改的**delta 更新规则** (delta update rule)。我们观察到这两种机制是互补的——门控能够实现快速内存擦除，而 delta 规则则促进定向更新。基于这一洞察，我们引入了**门控 delta 规则** (gated delta rule)，并开发了针对现代硬件优化的并行训练算法。我们提出的架构 **Gated DeltaNet** 在语言建模、常识推理、上下文检索、长度外推和长上下文理解等多项基准测试中，一致性地超越了 Mamba2 和 DeltaNet 等现有模型。我们进一步通过开发混合架构来提升性能，该混合架构将 Gated DeltaNet 层与滑动窗口注意力或 Mamba2 层相结合，同时实现了更高的训练效率和更优的任务性能。

代码: <https://github.com/NVlabs/GatedDeltaNet>

通俗解释

这篇论文提出了一个叫做 **Gated DeltaNet** 的新模型。要理解它，可以打这样一个比方：想象你的大脑是一块有限大小的白板（隐状态/内存）。在阅读一段很长的文章时：

- **Mamba2 的门控机制**像一块大橡皮——可以快速地白板上的所有内容同时变淡（整体遗忘），但不能精确地只擦掉某一个词。
- **DeltaNet 的 delta 规则**像一支精细的修正笔——可以精确地替换白板上的某个特定内容，但无法一次性清空整块白板。

Gated DeltaNet 的核心思想就是：**把大橡皮和修正笔结合起来使用**。需要清空时用橡皮 ($\alpha_t \rightarrow 0$)，需要精确修改时用修正笔 ($\alpha_t \rightarrow 1$)。

论文还解决了一个工程问题：如何让这种新的更新规则能够在 GPU 上高效并行训练，最终在各种基准测试中取得了全面超越。

关键概念/总结

本文三大核心贡献：

1. **门控 delta 规则**：将 Mamba2 的门控遗忘与 DeltaNet 的精确更新统一到一个简洁的递推公式中。
2. **硬件高效的并行训练算法**：基于扩展的 WY 表示和分块并行策略，使训练可以充分利用 GPU 张量核心。
3. **混合架构**：通过与滑动窗口注意力、Mamba2 层的堆叠组合，进一步提升性能和训练吞吐量。

2 第1节：引言 (Introduction)

原文翻译

Transformer 架构凭借其有效的注意力机制，显著推动了大语言模型 (LLM) 的能力发展，在广泛的任务中展现了卓越的性能。该机制擅长精确的序列建模，并在训练期间利用现代 GPU 的并行处理能力。然而，自注意力组件随序列长度呈二次方增长，导致训练和推理的计算需求巨大。

【引用 Katharopoulos et al., 2020】

提出了线性 Transformer，用核化的点积注意力替代 softmax 注意力，是线性注意力的奠基工作。

为缓解这些问题，研究者探索了线性 Transformer 等替代方案，它们将传统的基于 softmax 的注意力替换为基于核化点积的线性注意力，通过将其重新表述为具有矩阵值状态的线性 RNN，大幅降低了推理期间的内存需求。虽然早期版本的线性 Transformer 在语言建模任务中的表现不及标准 Transformer，但近期的增强——如引入类似 LSTM 的数据依赖门控机制，如 GLA 【Yang et al., 2023】

GLA（门控线性注意力）：提出了带有细粒度衰减的门控线性注意力机制及其硬件高效算法。

和 Mamba2 【Dao & Gu, 2024】

Mamba2：揭示了状态空间模型与线性注意力之间的对偶性（SSD），提出了高效的训练算法。

——已显示出令人期待的改进。然而，在管理长序列上的信息方面仍然存在挑战，特别是在上下文检索任务中，传统 Transformer 仍保持优势。

这种现象并不令人惊讶：线性 Transformer 可以被解释为实现了一个基于外积的键值关联记忆，类似于张量积表示。然而，它们能存储的正交键值对数量受到模型维度的限制。当序列长度超过该维度时，“内存碰撞”就不可避免，从而阻碍了精确检索。

Mamba2 通过引入简单的门控更新规则 $\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ 来解决这一限制，该规则在每个时间步以动态比率 $\alpha_t \in (0, 1)$ 统一衰减所有键值关联。然而，这种方法不考虑不同键值关联的重要性差异，可能导致内存利用效率低下。如果模型需要遗忘某个特定的键值关联，所有键值关联都会被同等程度地遗忘，使得该过程缺乏针对性和效率。

相比之下，带有 delta 规则的线性 Transformer，即 **DeltaNet**，通过以序列方式（软）替换旧的键值对为新的传入键值对来选择性更新内存。这种方法在合成基准的上下文检索中表现出色。然而，由于该过程每次只修改单个键值对，模型缺乏快速清除过时或不相关信息的能力，特别是在需要擦除先前数据的上下文切换期间。因此，DeltaNet 在实际任务中的表现一般，这很可能是由于缺乏稳健的内存清除机制。

认识到门控更新规则和 delta 规则在内存管理中的互补优势，我们提出了**门控 delta 规则**（gated delta rule），一种简洁而直观的机制，结合了两种方法的优势。该统一规则能够实现灵活的内存控制：通过设置 $\alpha_t \rightarrow 0$ 来迅速清除内存，同时通过设置 $\alpha_t \rightarrow 1$ （有效切换到纯 delta 规则）来选择性地更新特定内容而不影响其他信息。

剩余的挑战在于以硬件高效的方式实现门控 delta 规则。基于 Yang et al. (2024) 使用 WY 表示并行化 delta 规则计算的高效算法，我们仔细扩展了其方法以整合门控项。我们的扩展保留了分块并行的优势，使得硬件高效训练成为可能。

我们最终的架构 Gated DeltaNet 在语言建模、常识推理、上下文检索、长度外推和长上下文理解等全面的基准测试套件中，一致性地超越了 Mamba2 和 DeltaNet。在此基础上，我们还开发了混合架构，战略性地结合 Gated DeltaNet 层与滑动窗口注意力或 Mamba2 层，进一步提升了训练效率和模型性能。

通俗解释

引言部分讲了一个完整的”问题→已有方案的不足→本文的解决方案”的故事：

问题：标准 Transformer 的自注意力计算量随序列长度呈二次方增长 ($O(L^2)$)，在很长的文本上效率很低。

已有方案及其不足：

- **线性 Transformer：**把二次复杂度降到了线性，但性能差距大——因为它的”白板”能记住的正交信息有限。
- **Mamba2（加了门控的线性注意力）：**通过一个”整体衰减”来遗忘旧信息。问题

是”一刀切”——想忘掉 A 时，B、C、D 也会被同时减弱。

- **DeltaNet**（加了 delta 规则的线性注意力）：能精确替换某个特定键值对。问题是“太慢”——无法快速清空整个内存来适应新话题。

本文方案：将门控（整体遗忘能力）和 delta 规则（精确更新能力）结合，得到一个两全其美的方案。

注意事项

线性 Transformer ≠ 线性复杂度的 Transformer。”线性 Transformer”特指将 softmax 注意力替换为核化点积形式的模型，其关键特性是可以写成线性递推形式 $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ 。这里的”线性”指的是递推中的线性运算，不仅仅是计算复杂度。

3 第2节：预备知识 (Preliminary)

3.1 2.1 Mamba2：带衰减的线性注意力

原文翻译

已知线性 Transformer（排除归一化和 query/key 激活函数时）可以表述为以下线性递推：

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \in \mathbb{R}^{d_v \times d_k}, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \in \mathbb{R}^{d_v} \quad (1)$$

其中 d_k 和 d_v 分别表示 query/key 和 value 的（头）维度。通过展开递推，我们可以将其表示为向量形式（左）和矩阵形式（右）：

$$\mathbf{o}_t = \sum_{i=1}^t (\mathbf{v}_i \mathbf{k}_i^\top) \mathbf{q}_t = \sum_{i=1}^t \mathbf{v}_i (\mathbf{k}_i^\top \mathbf{q}_t) \in \mathbb{R}^{d_v}, \quad \mathbf{O} = (\mathbf{Q} \mathbf{K}^\top \odot \mathbf{M}) \mathbf{V} \in \mathbb{R}^{L \times d_v} \quad (2)$$

其中 L 是序列长度， $\mathbf{M} \in \mathbb{R}^{L \times L}$ 是因果掩码，定义为当 $i < j$ 时 $\mathbf{M}_{ij} = 0$ ，否则为 1。然而，这种原始的线性注意力在语言建模中远远落后于 Transformer。为解决这一问题，通常添加衰减项来遗忘历史信息。以 Mamba2 为例，它可以表示为以下线性递推（直到特定参数化）：

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \quad (3)$$

其中 $\alpha_t \in (0, 1)$ 是一个随 t 变化的数据依赖的标量衰减项。

定义累积衰减乘积 $\gamma_j = \prod_{i=1}^j \alpha_i$ ，通过展开递推，我们可以将结果表示为向量形式（左）和矩阵并行形式（右）：

$$\mathbf{o}_t = \sum_{i=1}^t \left(\frac{\gamma_t}{\gamma_i} \mathbf{v}_i \mathbf{k}_i^\top \right) \mathbf{q}_t, \quad \mathbf{O} = ((\mathbf{Q} \mathbf{K}^\top) \odot \mathbf{\Gamma}) \mathbf{V} \quad (4)$$

这里， $\mathbf{\Gamma} \in \mathbb{R}^{L \times L}$ 是衰减感知的因果掩码，其中当 $i \geq j$ 时 $\mathbf{\Gamma}_{ij} = \frac{\gamma_i}{\gamma_j}$ ，否则 $\mathbf{\Gamma}_{ij} = 0$ 。递推形式和并行形式之间的等价性也被称为 Mamba2 中描述的状态空间对偶性（SSD）。

通俗解释

这部分介绍了线性注意力的两种等价形式：

递推形式（适合推理）：就像一个持续更新的”记忆矩阵” $\mathbf{S}$ 。每来一个新的 token，就把

它的 value 和 key 的外积加到记忆里。查询时用 query 去查这个矩阵。

并行形式（适合训练）：本质上就是标准注意力公式 $\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V}$ ，去掉了 softmax。这样就可以直接用矩阵乘法并行计算整个序列。

Mamba2 的改进是加了一个衰减因子 α_t ——每一步都把旧记忆乘以一个小于 1 的数，让旧信息逐渐褪色。这就像记忆会随时间自然淡化一样。

数学推导：线性注意力递推展开

以最简单的线性注意力为例，展开递推过程：

符号定义：

- $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ ：时间步 t 的隐状态矩阵
- $\mathbf{k}_t \in \mathbb{R}^{d_k}$ ：时间步 t 的 key 向量
- $\mathbf{v}_t \in \mathbb{R}^{d_v}$ ：时间步 t 的 value 向量
- $\mathbf{q}_t \in \mathbb{R}^{d_k}$ ：时间步 t 的 query 向量

逐步展开：

$$\mathbf{S}_1 = \mathbf{S}_0 + \mathbf{v}_1 \mathbf{k}_1^\top = \mathbf{v}_1 \mathbf{k}_1^\top \quad (\text{假设 } \mathbf{S}_0 = \mathbf{0})$$

$$\mathbf{S}_2 = \mathbf{S}_1 + \mathbf{v}_2 \mathbf{k}_2^\top = \mathbf{v}_1 \mathbf{k}_1^\top + \mathbf{v}_2 \mathbf{k}_2^\top$$

$$\mathbf{S}_t = \sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top$$

输出为：

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t = \left(\sum_{i=1}^t \mathbf{v}_i \mathbf{k}_i^\top \right) \mathbf{q}_t = \sum_{i=1}^t \mathbf{v}_i \underbrace{(\mathbf{k}_i^\top \mathbf{q}_t)}_{\text{标量: key-query 相似度}}$$

这就是一个加权求和，权重是 key 和 query 的点积——与标准注意力的区别在于这里没有 softmax 归一化。

数值例子 ($d_k = d_v = 2$, 序列长度 $T = 2$)：

设 $\mathbf{k}_1 = [1, 0]^\top$, $\mathbf{v}_1 = [3, 1]^\top$, $\mathbf{k}_2 = [0, 1]^\top$, $\mathbf{v}_2 = [2, 4]^\top$, $\mathbf{q}_2 = [0.5, 0.8]^\top$

$$\mathbf{S}_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$$

$$\mathbf{o}_2 = \mathbf{S}_2 \mathbf{q}_2 = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 3.1 \\ 3.7 \end{bmatrix}$$

Mamba2 的衰减版本：如果 $\alpha_2 = 0.9$ ，则：

$$\mathbf{S}_2 = 0.9 \cdot \mathbf{S}_1 + \mathbf{v}_2 \mathbf{k}_2^\top = 0.9 \cdot \begin{bmatrix} 3 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 2 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} 2.7 & 2 \\ 0.9 & 4 \end{bmatrix}$$

注意旧信息 (3, 0; 1, 0) 被均匀缩小为 (2.7, 0; 0.9, 0)——**所有旧记忆都被同等衰减了**。

原文翻译

分块训练 (Chunkwise training) 然而，递推和并行形式对于高效训练都不是理想的，这促使了分块并行形式的使用，以实现硬件高效的线性时间训练。

分块并行形式将输入和输出分割为大小为 C 的多个块，并基于前一个块的最终状态和当前块的 query/key/value 块来计算每个块的输出。记号约定： $\mathbf{Q}_{[t]}$ 为块 t 的 query 块， $\mathbf{q}_{[t]}^r$ 为

块 t 中第 r 个 query。块 t 的初始状态定义为 $\mathbf{S}_{[t]} := \mathbf{S}_{[t]}^0 = \mathbf{S}_{[t-1]}^C$ 。通过部分展开递推：

$$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]} + \sum_{i=1}^r \mathbf{v}_{[t]}^i \mathbf{k}_{[t]}^{i\top}, \quad \mathbf{o}_{[t]}^r = \mathbf{S}_{[t]} \mathbf{q}_{[t]}^r + \sum_{i=1}^r \mathbf{v}_{[t]}^i \left(\mathbf{k}_{[t]}^{i\top} \mathbf{q}_{[t]}^r \right) \quad (5)$$

等价的矩阵形式：

$$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]} + \mathbf{V}_{[t]}^\top \mathbf{K}_{[t]}, \quad \mathbf{O}_{[t]} = \mathbf{Q}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{M} \right) \mathbf{V}_{[t]} \quad (6)$$

上述方程富含矩阵乘法（matmuls），允许基于张量核心的硬件优化。此分块算法可以轻松扩展到带衰减的线性注意力：

$$\mathbf{S}_{[t+1]} = \overleftarrow{\mathbf{S}}_{[t]} + \mathbf{V}_{[t]}^\top \overrightarrow{\mathbf{K}}_{[t]}, \quad \mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{\Gamma}_{[t]} \right) \mathbf{V}_{[t]} \quad (7)$$

其中 $(\mathbf{\Gamma}_{[t]})_{ij} = \frac{\gamma_{[t]}^i}{\gamma_{[t]}^j}$, $\gamma_{[t]}^j = \prod_{j=tC+1}^{tC+j} \alpha_j$ 。^a

这里使用左箭头（ $\overleftarrow{\cdot}$ ）或右箭头（ $\overrightarrow{\cdot}$ ）分别表示衰减到每个块的第一个位置和最后一个位置的变量：

$$\begin{aligned} \overleftarrow{\mathbf{q}}_{[t]}^r &= \gamma_{[t]}^r \mathbf{q}_{[t]}^r && \text{（将每个向量衰减到块 } t \text{ 的第一个位置）} \\ \overrightarrow{\mathbf{k}}_{[t]}^r &= \frac{\gamma_{[t]}^C}{\gamma_{[t]}^r} \mathbf{k}_{[t]}^r && \text{（将每个向量衰减到块 } t \text{ 的最后一个位置）} \\ \overrightarrow{\mathbf{S}}_{[t]} &= \gamma_{[t]}^C \mathbf{S}_{[t]} && \text{（将状态矩阵在整个块 } t \text{ 上衰减）} \end{aligned} \quad (8)$$

^a这里我们稍微滥用了 γ 的记号，表示每个块内的累积乘积（从每个块的第一个位置分别开始），而不是整个序列的。

通俗解释

分块并行是一个非常巧妙的”两全其美”策略：

为什么需要它？

- **纯递推形式**：一步一步计算，无法并行，训练太慢。
- **纯并行形式**：需要计算完整的 $L \times L$ 注意力矩阵，当 L 很大时内存放不下。

分块并行的核心思想：把长度为 L 的序列切成 L/C 个小块，每块长 C （例如 $C = 64$ ）。

- **块内**：用并行形式（矩阵乘法），只需计算 $C \times C$ 的小注意力矩阵——很快，内存也够。
- **块间**：用递推形式，把上一个块的最终状态传递给下一个块。

这样，总复杂度是 $O(L \cdot C \cdot d)$ ，而且块内计算可以充分利用 GPU 的张量核心（Tensor Core）。

箭头符号的含义： $\overleftarrow{\mathbf{q}}$ 表示”向左看”（从当前位置衰减到块的起点）， $\overrightarrow{\mathbf{k}}$ 表示”向右看”（从当前位置衰减到块的终点）。这些衰减因子确保了块间传递状态时的数值正确性。

3.2 2.2 Delta Networks：带 Delta 规则的线性注意力

原文翻译

delta 更新规则 **动态地** 擦除与当前输入 key ($\mathbf{k}_t$) 关联的旧 value ($\mathbf{v}_t^{\text{old}}$)，并写入一个新 value

($\mathbf{v}_t^{\text{new}}$), 新 value 是当前输入 value 和旧 value 基于”写入强度” $\beta_t \in (0, 1)$ 的线性组合。^a

$$\begin{aligned}\mathbf{S}_t &= \mathbf{S}_{t-1} - \underbrace{(\mathbf{S}_{t-1}\mathbf{k}_t)\mathbf{k}_t^\top}_{\mathbf{v}_t^{\text{old}}} + \underbrace{(\beta_t\mathbf{v}_t + (1-\beta_t)\mathbf{S}_{t-1}\mathbf{k}_t)\mathbf{k}_t^\top}_{\mathbf{v}_t^{\text{new}}} \\ &= \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top) + \beta_t\mathbf{v}_t\mathbf{k}_t^\top\end{aligned}\quad (9)$$

如上所示, DeltaNet 实现了一阶线性递推, 其广义 Householder 转移矩阵为 $(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top)$ 。尽管展现了优越的关联记忆和语言建模性能, DeltaNet 由于计算效率低下而受到有限关注, 直到 Yang et al. (2024) 引入了硬件高效的分块训练算法。

^a可以将 β_t 设置在 $(0, 2)$ 以允许负特征值, 从而解锁 DeltaNet 的状态追踪能力。

数学推导: Delta 规则的直观理解

Delta 规则的更新可以分为三步理解:

第一步: 读出旧值。用当前 key 去查询记忆矩阵, 得到该 key 对应的旧 value:

$$\mathbf{v}_t^{\text{old}} = \mathbf{S}_{t-1}\mathbf{k}_t \in \mathbb{R}^{d_v}$$

第二步: 混合新旧值。将新输入的 value 和旧 value 按比例 β_t 混合:

$$\mathbf{v}_t^{\text{new}} = \beta_t\mathbf{v}_t + (1-\beta_t)\mathbf{v}_t^{\text{old}}$$

当 $\beta_t = 1$ 时完全用新值, 当 $\beta_t = 0$ 时完全保留旧值。

第三步: 替换更新。先擦除旧 value, 再写入新 value (都绑定到 key $\mathbf{k}_t$):

$$\begin{aligned}\mathbf{S}_t &= \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{old}}\mathbf{k}_t^\top + \mathbf{v}_t^{\text{new}}\mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} - (\mathbf{S}_{t-1}\mathbf{k}_t)\mathbf{k}_t^\top + (\beta_t\mathbf{v}_t + (1-\beta_t)\mathbf{S}_{t-1}\mathbf{k}_t)\mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1} - \mathbf{S}_{t-1}\mathbf{k}_t\mathbf{k}_t^\top + \beta_t\mathbf{v}_t\mathbf{k}_t^\top + \mathbf{S}_{t-1}\mathbf{k}_t\mathbf{k}_t^\top - \beta_t\mathbf{S}_{t-1}\mathbf{k}_t\mathbf{k}_t^\top \\ &= \mathbf{S}_{t-1}(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top) + \beta_t\mathbf{v}_t\mathbf{k}_t^\top\end{aligned}$$

关键区别: 与 Mamba2 的 $\mathbf{S}_t = \alpha_t\mathbf{S}_{t-1} + \mathbf{v}_t\mathbf{k}_t^\top$ 相比, DeltaNet 的转移矩阵是 $(\mathbf{I} - \beta_t\mathbf{k}_t\mathbf{k}_t^\top)$, 而不是 $\alpha_t\mathbf{I}$ 。前者是一个 Householder 矩阵——它只在 $\mathbf{k}_t$ 方向上进行衰减, 而保持其他方向不变。这就是”精确更新”的数学含义。

数值例子 ($d_k = d_v = 2, \beta_t = 0.8$):

$$\text{设 } \mathbf{S}_0 = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}, \mathbf{k}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ (已归一化)}, \mathbf{v}_1 = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

$$\text{旧值: } \mathbf{v}^{\text{old}} = \mathbf{S}_0\mathbf{k}_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

$$\text{转移矩阵: } \mathbf{I} - 0.8\mathbf{k}_1\mathbf{k}_1^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.8 \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.2 & 0 \\ 0 & 1 \end{bmatrix}$$

注意: 只有 $\mathbf{k}_1$ 方向 (第一行第一列) 被衰减了 ($1 \rightarrow 0.2$), 另一个方向 (第二行第二列) 保持不变!

$$\mathbf{S}_1 = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 0.2 & 0 \\ 0 & 1 \end{bmatrix} + 0.8 \begin{bmatrix} 5 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 0.6 & 2 \\ 0.2 & 4 \end{bmatrix} + \begin{bmatrix} 4 & 0 \\ 1.6 & 0 \end{bmatrix} = \begin{bmatrix} 4.6 & 2 \\ 1.8 & 4 \end{bmatrix}$$

观察第二列 ($\mathbf{k}$ 正交方向) 的值 (2, 4) 完全没变! 这就是 delta 规则的精确性。

原文翻译

分块并行形式。 通过部分展开递推，有：

$$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]} \underbrace{\left(\prod_{i=1}^r \mathbf{I} - \beta_{[t]}^i \mathbf{k}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \right)}_{:=\mathbf{P}_{[t]}^r} + \underbrace{\sum_{i=1}^r \left(\beta_{[t]}^i \mathbf{v}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \prod_{j=i+1}^r \left(\mathbf{I} - \beta_{[t]}^j \mathbf{k}_{[t]}^j \mathbf{k}_{[t]}^{j\top} \right) \right)}_{:=\mathbf{H}_{[t]}^r} \quad (10)$$

其中 $\mathbf{P}_{[t]}^j$ 涉及广义 Householder 矩阵的累积乘积，可通过经典的 WY 表示进行优化：

$$\mathbf{P}_{[t]}^r = \mathbf{I} - \sum_{i=1}^r \mathbf{w}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \in \mathbb{R}^{d_k \times d_k} \quad \mathbf{w}_{[t]}^r = \beta_{[t]}^r \left(\mathbf{k}_{[t]}^r - \sum_{i=1}^{r-1} \left(\mathbf{w}_{[t]}^i (\mathbf{k}_{[t]}^{i\top} \mathbf{k}_{[t]}^r) \right) \right) \in \mathbb{R}^{d_k} \quad (11)$$

类似地， $\mathbf{H}_{[t]}^r$ 可以表示为：

$$\mathbf{H}_{[t]}^r = \sum_{i=1}^r \mathbf{u}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \in \mathbb{R}^{d_v \times d_k} \quad \mathbf{u}_{[t]}^r = \beta_{[t]}^r \left(\mathbf{v}_{[t]}^r - \sum_{i=1}^{r-1} \left(\mathbf{u}_{[t]}^i (\mathbf{k}_{[t]}^{i\top} \mathbf{k}_{[t]}^r) \right) \right) \in \mathbb{R}^{d_v} \quad (12)$$

矩阵形式为： $\mathbf{P}_{[t]} = \mathbf{I} - \mathbf{W}_{[t]}^\top \mathbf{K}_{[t]}$ ， $\mathbf{H}_{[t]} = \mathbf{U}_{[t]}^\top \mathbf{K}_{[t]}$ 。通过 UT 变换，可进一步写为：

$$\mathbf{T}_{[t]} = \left[\mathbf{I} + \text{strictLower} \left(\text{diag}(\beta_{[t]}) \mathbf{K}_{[t]} \mathbf{K}_{[t]}^\top \right) \right]^{-1} \text{diag}(\beta_{[t]}) \in \mathbb{R}^{C \times C} \quad (13)$$

$$\mathbf{W}_{[t]} = \mathbf{T}_{[t]} \mathbf{K}_{[t]} \in \mathbb{R}^{C \times d_k}, \quad \mathbf{U}_{[t]} = \mathbf{T}_{[t]} \mathbf{V}_{[t]} \in \mathbb{R}^{C \times d_v} \quad (14)$$

代回后得到 DeltaNet 的硬件高效分块算法：

$$\mathbf{S}_{[t+1]} = \mathbf{S}_{[t]} \mathbf{P}_{[t]} + \mathbf{H}_{[t]} = \mathbf{S}_{[t]} + \left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}^\top \right)^\top \mathbf{K}_{[t]} \in \mathbb{R}^{d_v \times d_k} \quad (15)$$

$$\mathbf{O}_{[t]} = \mathbf{Q}_{[t]} \mathbf{S}_{[t]}^\top + (\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{M}) \left(\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}^\top \right) \in \mathbb{R}^{C \times d_v} \quad (16)$$

通俗解释

这部分展示了如何把 DeltaNet 的逐步递推”打包”成可以并行的矩阵运算。

核心难题：DeltaNet 的转移矩阵 $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ 在每一步都不同（因为 $\mathbf{k}_t$ 和 β_t 随输入变化），所以不能像 Mamba2 那样简单地用标量衰减来处理。

WY 表示是数值线性代数中的经典技巧：它可以把多个 Householder 矩阵的连乘转化为”单位矩阵减去一个低秩矩阵”的形式。这个转化让原本的连乘变成了矩阵乘法，可以利用 GPU 的张量核心加速。

最终的分块算法（方程 15-16）有两个关键部分：

- **状态更新（15）**：块间传递隐状态。
- **输出计算（16）**：分为”来自之前块的贡献”（第一项）和”来自当前块内的贡献”（第二项，通过 $C \times C$ 的掩码注意力计算）。

关键概念/总结

预备知识总结：

1. **线性注意力**=矩阵值递推 $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ ，有递推和并行两种等价形式。
2. **Mamba2**在线性注意力上添加标量衰减 α_t ，实现整体遗忘。

3. DeltaNet使用 Householder 转移矩阵 $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ ，实现精确的单键更新。
4. 分块并行是让这些递推模型高效训练的关键——块内并行，块间递推。
5. WY 表示是 DeltaNet 能够高效并行的数学基础。

4 第3节: Gated Delta Networks

4.1 3.1 公式化: 门控 Delta 规则

原文翻译

提出的门控 delta 规则简洁而有效:

$$\mathbf{S}_t = \mathbf{S}_{t-1} (\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \quad (17)$$

其中数据依赖的门控项 $\alpha_t \in (0, 1)$ 控制状态衰减。这一公式统一了门控机制和 delta 规则的优势: 门控项实现了自适应的内存管理, 而 delta 更新结构促进了有效的键值关联学习。

我们通过 Longhorn 引入的在线学习框架对门控 delta 规则进行了形式化分析。在该框架中, 递推状态更新作为在线学习问题的闭式解出现, 如表 1 所示。近期的线性 RNN 架构通常在其在在线学习目标中引入正则化项, 以防止状态与先前值的偏差过大, 从而实现记忆保持。然而, 当状态因信息饱和时, 这种保持机制会变得有问题。在这种情况下, 每个状态将编码多条信息的叠加, 使得精确检索变得困难。

为解决这一限制, Mamba2 和 Gated DeltaNet 引入了自适应缩放因子 α_t , 松弛了正则化项, 允许 $\mathbf{S}_t$ 和 $\mathbf{S}_{t-1}$ 之间的受控偏差。这种修改通过选择性遗忘实现了动态内存管理。

另一方面, 线性注意力和 Mamba2 使用简单的负内积损失 $-\langle \mathbf{S}_t \mathbf{k}_t, \mathbf{v}_t \rangle$, 而 Longhorn 使用更具表现力的在线回归目标 $\|\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t\|^2$ 以更好地建模键值关联。Longhorn 的更新规则与 delta 更新规则密切相关,^a这表明 (门控) delta 规则在上下文关联检索中优于 Mamba2。

从快速权重编程和测试时训练以及回归的角度来看, 隐状态 $\mathbf{S}$ 可以被解释为 (快速) 权重矩阵, delta 规则通过测试时随机梯度下降 (SGD) 优化在线回归目标 $\mathcal{L}(\mathbf{S}_t) = \frac{1}{2} \|\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t\|^2$:

$$\mathbf{S}_{t+1} = \mathbf{S}_t - \beta_t \nabla \mathcal{L}(\mathbf{S}_t) = \mathbf{S}_t - \beta_t (\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top = \mathbf{S}_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \quad (18)$$

其中 β_t 表示 (自适应) 学习率。从这个角度看, 门控 delta 规则可以被视为在 SGD 更新中引入了自适应权重衰减项 α_t 。

^a理论区别在于优化方法: Longhorn 使用隐式在线学习推导闭式全局最优更新, 而 DeltaNet 通过单步显式梯度下降优化相同目标。

数学推导: 门控 Delta 规则的 SGD/权重衰减视角

步骤 1: 标准 delta 规则 = 在线回归的 SGD

目标函数: $\mathcal{L}(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} \mathbf{k}_t - \mathbf{v}_t\|^2$

对 $\mathbf{S}$ 求梯度:

$$\nabla_{\mathbf{S}} \mathcal{L} = \frac{\partial}{\partial \mathbf{S}} \frac{1}{2} \|\mathbf{S} \mathbf{k}_t - \mathbf{v}_t\|^2 = (\mathbf{S} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top$$

SGD 更新:

$$\begin{aligned} \mathbf{S}_{t+1} &= \mathbf{S}_t - \beta_t \nabla_{\mathbf{S}} \mathcal{L} = \mathbf{S}_t - \beta_t (\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top \\ &= \mathbf{S}_t - \beta_t \mathbf{S}_t \mathbf{k}_t \mathbf{k}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \\ &= \mathbf{S}_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \quad \checkmark \end{aligned}$$

步骤 2: 门控 delta 规则 = 带权重衰减的 SGD

在深度学习中，带权重衰减的 SGD 为：

$$\mathbf{S}_{t+1} = (1 - \lambda)\mathbf{S}_t - \beta_t \nabla \mathcal{L}(\mathbf{S}_t)$$

类比地，设 $(1 - \lambda) = \alpha_t$ ：

$$\begin{aligned}\mathbf{S}_{t+1} &= \alpha_t \mathbf{S}_t - \beta_t (\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top \\ &= \alpha_t \mathbf{S}_t - \beta_t \mathbf{S}_t \mathbf{k}_t \mathbf{k}_t^\top + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \\ &= \mathbf{S}_t \cdot \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top \quad \checkmark\end{aligned}$$

这正好就是公式 (17)！所以 α_t 扮演的角色就是深度学习中的**权重衰减**——当 $\alpha_t < 1$ 时，模型倾向于让隐状态更小，相当于”淡忘”旧信息。

原文翻译

表 1 展示了不同线性 RNN 模型及其对应的在线学习目标。

Table 1: 不同线性 RNN 模型及其对应的在线学习目标对比（来自 Longhorn 框架）。

方法	在线学习目标	在线更新规则
LA	$\ \mathbf{S}_t - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \mathbf{v}_t \rangle$	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^T$
Mamba2	$\ \mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \mathbf{v}_t \rangle$	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^T$
Longhorn	$\ \mathbf{S}_t - \mathbf{S}_{t-1}\ _F^2 - \beta_t \ \mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t\ ^2$	$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \epsilon_t \mathbf{k}_t \mathbf{k}_t^T) + \epsilon_t \mathbf{v}_t \mathbf{k}_t^T$
DeltaNet	$\ \mathbf{S}_t - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$	$\mathbf{S}_t = \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^T) + \beta_t \mathbf{v}_t \mathbf{k}_t^T$
Gated DeltaNet	$\ \mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$	$\mathbf{S}_t = \mathbf{S}_{t-1} (\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^T)) + \beta_t \mathbf{v}_t \mathbf{k}_t^T$

图表解读

表 1 解读：

这张表清晰地展示了五种模型的递进关系：

- **LA（线性注意力）**：最简单，只是不断累加，没有遗忘。
- **Mamba2**：在 LA 基础上加了 α_t 衰减（整体遗忘），但目标函数仍是简单的内积匹配。
- **Longhorn**：使用了更好的回归损失 $\|\mathbf{S}_t \mathbf{k}_t - \mathbf{v}_t\|^2$ （精确匹配），但没有 α_t 衰减。
- **DeltaNet**：使用了类似的精确匹配损失，但没有 α_t 衰减。
- **Gated DeltaNet**：同时有 α_t 衰减和精确匹配——集大成者。

4.2 3.2 案例研究：大海捞针 (S-NIAH)

原文翻译

为更好地理解 delta 规则和门控规则之间的互补优势，我们在 RULER 的大海捞针（S-NIAH）基准测试套件上进行案例研究，其中一个键值对作为上下文中的”针”，模型在给定 key 时必须回忆对应的 value。

衰减损害记忆保持。 在最简单的 S-NIAH-1 设置中，使用重复的合成上下文，模型记忆最少量的信息，测试长期保持能力。DeltaNet 在所有序列长度上都达到了近乎完美的性能。

Mamba2 在超过 2K 序列后显著退化，因为它衰减历史信息太快，而 Gated DeltaNet 的退化得益于 delta 规则的使用而较轻。

门控促进过滤。 在使用真实世界文章上下文的 S-NIAH-2/3 中，模型存储所有潜在相关的信息，测试高效的内存管理。在固定状态大小下，缺乏清除能力导致内存碰撞——信息变得叠加且无法区分。DeltaNet 在较长序列中性能显著下降，因为其内存清除能力差。Mamba2 和 Gated DeltaNet 通过过滤不相关信息的门控机制保持了更好的性能。

Delta 规则帮助记忆。 在 S-NIAH-3 中，value 从数字变为 UUID，测试复杂模式的记忆能力。Mamba2 的性能快速下降，而 Gated DeltaNet 表现更好，验证了 delta 规则确实具有更好的记忆能力。

Table 2: 1.3B 模型在 S-NIAH 基准测试套件上的零样本性能对比。

Model	S-NIAH-1 (密钥检索)				S-NIAH-2 (数字大海捞针)				S-NIAH-3 (UUID 大海捞针)		
	1K	2K	4K	8K	1K	2K	4K	8K	1K	2K	4K
DeltaNet	97.4	96.8	99.0	98.8	98.4	45.6	18.6	14.4	85.2	47.0	22.4
Mamba2	99.2	98.8	65.4	30.4	99.4	98.8	56.2	17.0	64.4	47.6	4.6
Gated DeltaNet	98.4	88.4	91.4	91.8	100.0	99.8	92.2	29.6	86.6	84.2	27.6

图表解读

表 2 解读：

这张表是论文的核心实验之一，清晰地展示了三种模型的互补性：

S-NIAH-1（简单场景：重复合成上下文）：DeltaNet 几乎满分（98–99%），因为它精确保持了记忆。Mamba2 在长序列（4K/8K）上暴跌（65%/30%），因为衰减太快导致针被“遗忘”了。Gated DeltaNet 保持在 88–92%。

S-NIAH-2（真实文章上下文 + 数字针）：DeltaNet 暴跌（4K 仅 18.6%），因为它不能有效过滤大量无关信息，导致内存碰撞。Mamba2 表现好很多（4K 56.2%），因为门控可以过滤。**Gated DeltaNet 大幅领先**（4K 92.2%）。

S-NIAH-3（UUID 值——更复杂的模式）：Mamba2 的记忆力不足（4K 4.6%），DeltaNet 也不佳（4K 22.4%）。Gated DeltaNet 再次领先（4K 27.6%）。

结论：Gated DeltaNet 确实结合了两种优势——gate 帮助过滤，delta 帮助精确记忆。

4.3 3.3 算法：硬件高效的分块训练

原文翻译

在本小节中，我们推导 Gated DeltaNet 的硬件高效分块训练算法。通过部分展开公式 (17) 中的递推：

$$\mathbf{S}_{[t]}^r = \mathbf{S}_{[t]} \left(\underbrace{\prod_{i=1}^r \alpha_{[t]}^i \left(\mathbf{I} - \beta_{[t]}^i \mathbf{k}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \right)}_{:= \mathbf{F}_{[t]}^r} \right) + \underbrace{\sum_{i=1}^r \left(\beta_{[t]}^i \mathbf{v}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \prod_{j=i+1}^r \alpha_{[t]}^j \left(\mathbf{I} - \beta_{[t]}^j \mathbf{k}_{[t]}^j \mathbf{k}_{[t]}^{j\top} \right) \right)}_{:= \mathbf{G}_{[t]}^r} \quad (19)$$

容易看出 $\mathbf{F}_{[t]}^r = \gamma_{[t]}^r \mathbf{P}_{[t]}^r = \overline{\mathbf{P}_{[t]}^r}$ 。对于 $\mathbf{G}_{[t]}^r$ ，我们调整公式 (12) 如下：

$$\mathbf{G}_{[t]}^r = \sum_{i=1}^r \frac{\gamma_{[t]}^r}{\gamma_{[t]}^i} \tilde{\mathbf{u}}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \quad \tilde{\mathbf{u}}_{[t]}^r = \beta_{[t]}^r \left(\mathbf{v}_{[t]}^r - \sum_{i=1}^{r-1} \left(\tilde{\mathbf{u}}_{[t]}^i \left(\frac{\gamma_{[t]}^r}{\gamma_{[t]}^i} \mathbf{k}_{[t]}^{i\top} \mathbf{k}_{[t]}^r \right) \right) \right) \quad (20)$$

通过 UT 变换，矩阵形式为：

$$\widetilde{\mathbf{U}}_{[t]} = \left[\mathbf{I} + \text{strictLower} \left(\text{diag}(\beta_{[t]}) (\mathbf{\Gamma}_{[t]} \odot \mathbf{K}_{[t]} \mathbf{K}_{[t]}^\top) \right) \right]^{-1} \text{diag}(\beta_{[t]}) \mathbf{V}_{[t]} \in \mathbb{R}^{C \times d_v} \quad (21)$$

最终的硬件高效分块算法：

$$\mathbf{S}_{[t+1]} = \overrightarrow{\mathbf{S}}_{[t]} + \left(\widetilde{\mathbf{U}}_{[t]} - \overleftarrow{\mathbf{W}}_{[t]} \mathbf{S}_{[t]}^\top \right)^\top \overrightarrow{\mathbf{K}}_{[t]} \in \mathbb{R}^{d_v \times d_k} \quad (22)$$

$$\mathbf{O}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + (\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{M}) \left(\widetilde{\mathbf{U}}_{[t]} - \overleftarrow{\mathbf{W}}_{[t]} \mathbf{S}_{[t]}^\top \right) \in \mathbb{R}^{C \times d_v} \quad (23)$$

其中 $\overleftarrow{\mathbf{q}}_{[t]}^r = \gamma_{[t]}^r \mathbf{q}_{[t]}^r$, $\overleftarrow{\mathbf{w}}_{[t]}^r = \gamma_{[t]}^r \mathbf{w}_{[t]}^r$, $\overrightarrow{\mathbf{k}}_{[t]}^r = \frac{\gamma_{[t]}^c}{\gamma_{[t]}^r} \mathbf{k}_{[t]}^r$, $\overrightarrow{\mathbf{S}}_{[t]} = \gamma_{[t]}^c \mathbf{S}_{[t]}$ 。

通俗解释

这部分是论文最核心的技术贡献之一。关键问题是：**Gated DeltaNet 的递推比 DeltaNet 更复杂（多了 α_t 门控项），如何设计高效的并行训练算法？**

答案是：巧妙地将门控因子”吸收”到 DeltaNet 已有的 WY 表示框架中。具体地：

1. $\mathbf{F}_{[t]}^r$ 部分：原本 DeltaNet 有 Householder 矩阵的乘积 $\mathbf{P}_{[t]}^r$ ，现在每个乘积项前面多了 $\alpha_{[t]}^i$ 。由于 α 是标量，这些可以合并为累积乘积 $\gamma_{[t]}^r$ ，所以 $\mathbf{F}_{[t]}^r = \gamma_{[t]}^r \mathbf{P}_{[t]}^r$ ——只是在 DeltaNet 的 $\mathbf{P}$ 上乘了一个标量！
2. $\mathbf{G}_{[t]}^r$ 部分：需要更仔细地处理。原 DeltaNet 的 $\mathbf{u}$ 递推需要加入衰减比 $\gamma_{[t]}^r / \gamma_{[t]}^i$ 作为修正因子。这些衰减比可以预计算。
3. **UT 变换**中，只需将 $\mathbf{K} \mathbf{K}^\top$ 替换为 $\mathbf{\Gamma} \odot \mathbf{K} \mathbf{K}^\top$ （逐元素乘以衰减矩阵）。

最终公式 (22)-(23) 和 DeltaNet 的 (15)-(16) 结构完全相同，只是各变量被加上了衰减因子的修饰（箭头符号）。因此，**Gated DeltaNet 的训练开销与 DeltaNet 几乎相同！**

4.4 3.4 Gated Delta Networks 和混合模型

原文翻译

Token 混合器块。 基本的 Gated DeltaNet 遵循 Llama 的宏观架构，将 token 混合器层与 SwiGLU MLP 层堆叠，但将自注意力替换为门控 delta 规则的 token 混合。图 1（右）展示了其块设计。对于门控 delta 规则（公式 17），query、key 和 value $\{\mathbf{q}, \mathbf{k}, \mathbf{v}\}$ 通过线性投影、短卷积和 SiLU 生成， $\mathbf{q}, \mathbf{k}$ 应用 L2 归一化以确保训练稳定性。 α, β 仅使用线性投影。^a 按照 RetNet 的做法，输出经过归一化和门控处理后再应用输出投影。

混合模型。 线性 Transformer 在建模局部偏移和比较方面有局限性，其固定状态大小使得检索任务困难。遵循 Griffin 和 Samba 等近期混合架构，我们将线性递推层与滑动窗口注意力（SWA）结合，得到 **GatedDeltaNet-H1**。我们还堆叠 Mamba2、GatedDeltaNet 和 SWA，得到 **GatedDeltaNet-H2**。

^a我们使用 Mamba2 的参数化来生成 α ，但为简洁起见省略了细节。

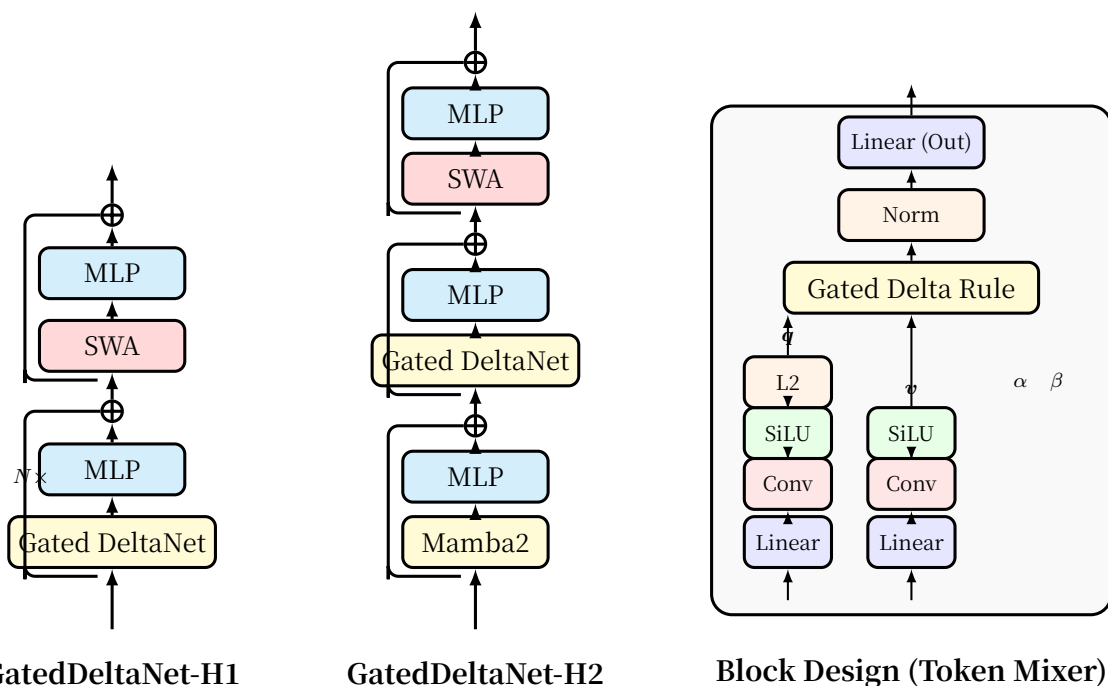


Figure 1: Gated DeltaNet 的（混合）架构与块设计示意图。H1 使用 Gated DeltaNet + SWA 模式，H2 使用 Mamba2 + Gated DeltaNet + SWA 模式。块设计中，query/key 经过线性投影→短卷积→SiLU→L2归一化；value 经过线性投影→短卷积→SiLU； α/β 仅使用线性投影；输出经过归一化和门控后投影。

图表解读

图 1 解读：

左：GatedDeltaNet-H1（两层循环）——每个 $N \times$ 重复单元包含 Gated DeltaNet + MLP + 残差，然后是 SWA + MLP + 残差。SWA（滑动窗口注意力）负责局部交互，Gated DeltaNet 负责全局/长距离交互。

中：GatedDeltaNet-H2（三层循环）——加入了 Mamba2 层，形成 Mamba2 → Gated DeltaNet → SWA 的三段式结构。实验证明这个顺序效果最好（见消融实验）。

右：块设计——展示了 Gated DeltaNet 内部的数据流。关键设计选择：L2 归一化用于 q, k 以稳定训练（因为 delta 规则中 kk^T 的数值稳定性依赖于 key 的归一化）；短卷积为序列引入局部上下文；SiLU 激活增加非线性表现力。

5 第4节：实验 (Experiments)

原文翻译

实验设置 我们的实验涵盖了对近期最先进架构的全面比较，包括纯 Transformer 模型、基于 RNN 的方法和混合架构。基线包括：RetNet、HGRN2、Mamba、Mamba2、Samba 和 DeltaNet。

为公平比较，所有模型在相同条件下训练：1.3B 参数规模，从 FineWeb-Edu 数据集采样 100B token。使用 AdamW 优化器，峰值学习率 $4e-4$ ，权重衰减 0.1，梯度裁剪 1.0。学习率遵循余弦退火调度，预热期为 1B token，批量大小 0.5M token。所有模型使用 Llama2 分词器（词汇量 32,000）。训练长度设为 4K token，Samba 和混合模型使用 2K 滑动窗口。

关键概念/总结

实验设置的公平性非常重要：所有模型使用完全相同的数据集（FineWeb-Edu 100B token）、相同的超参数（AdamW, lr=4e-4）、相同的分词器（Llama2）和相同的模型规模（1.3B），唯一变量就是架构设计。这种控制变量的严格实验设置让结论更加可靠。

5.1 常识推理

原文翻译

在表 3 中，我们展示了 1.3B 参数模型在常识推理基准上的语言建模困惑度和零样本准确率。Gated DeltaNet 在两种规模上都一致地超越了包括 RetNet、HGRN2、Mamba、Mamba2 和 DeltaNet 在内的其他线性模型。混合变体进一步提升了性能。

Table 3: 1.3B 模型在语言建模和零样本常识推理上的性能对比。加粗表示最优，下划线表示次优。

模型	Wiki. ppl↓	LMB. ppl↓	LMB. acc↑	PIQA acc↑	Hella. acc_n↑	Wino. acc↑	ARC-e acc↑	ARC-c acc_n↑	SIQA acc↑	BoolQ acc↑	Avg.
递推模型											
RetNet	19.08	17.27	40.52	70.07	49.16	54.14	67.34	33.78	40.78	60.39	52.02
HGRN2	19.10	17.69	39.54	70.45	49.53	52.80	69.40	35.32	<u>40.63</u>	56.66	51.79
Mamba	17.92	15.06	43.98	71.32	52.91	52.95	69.52	35.40	37.76	61.13	53.12
Mamba2	<u>16.56</u>	<u>12.56</u>	<u>45.66</u>	<u>71.87</u>	<u>55.67</u>	<u>55.24</u>	72.47	<u>37.88</u>	40.20	60.13	<u>54.89</u>
DeltaNet	17.71	16.88	42.46	70.72	50.93	53.35	68.47	35.66	40.22	55.29	52.14
Gated DeltaNet	16.42	12.17	46.65	72.25	55.76	57.45	<u>71.21</u>	38.39	<u>40.63</u>	60.24	55.32
注意力或混合模型											
Transformer++	18.53	18.32	42.60	70.02	50.23	53.51	68.83	35.10	40.66	57.09	52.25
Samba	16.13	13.29	44.94	70.94	53.42	55.56	68.81	36.17	39.96	<u>62.11</u>	54.00
GDN-H1	<u>16.07</u>	12.12	<u>47.73</u>	72.57	<u>56.53</u>	58.40	<u>71.75</u>	40.10	<u>41.40</u>	63.21	56.40
GDN-H2	15.91	<u>12.55</u>	48.76	<u>72.19</u>	56.88	<u>57.77</u>	71.33	<u>39.07</u>	41.91	61.55	<u>56.18</u>

图表解读

表 3 解读：

核心发现：

在递推模型中，Gated DeltaNet 以 55.32% 的平均准确率和 16.42/12.17 的困惑度胜出。相比之下，之前最强的 Mamba2 为 54.89%。DeltaNet 仅为 52.14%，说明单独的 delta 规则在实际语言建模任务中并非最优。

在混合模型中，GDN-H1 达到 56.40%，GDN-H2 达到 56.18%，均大幅超越 Samba（54.00%）和纯 Transformer++（52.25%）。

关键观察：纯 Transformer++ 的性能甚至**低于**多数递推模型！这可能因为在只有 100B token 训练下，1.3B 参数的 Transformer 还未达到最优性能，而递推模型的归纳偏差在小数据量下更有帮助。

5.2 真实世界上下文检索

Table 4: 2K token 截断输入上的真实世界检索任务准确率。

模型	SWDE	SQD	FDA	TQA	NQ	Drop	Avg
递推模型							
RetNet	14.0	28.5	7.0	54.4	16.2	17.3	22.9
HGRN2	8.3	25.3	4.8	51.2	14.2	16.9	20.1
Mamba	9.8	25.8	3.7	54.3	14.9	17.4	21.0
Mamba2	<u>19.1</u>	<u>33.6</u>	25.3	61.0	20.8	<u>19.2</u>	<u>29.8</u>
DeltaNet	17.9	30.9	18.4	53.9	17.3	18.6	26.2
Gated DeltaNet	25.4	34.8	<u>23.7</u>	<u>60.0</u>	<u>20.0</u>	19.8	30.6
注意力或混合模型							
Transformer++	29.5	38.0	52.2	58.3	22.5	21.6	37.0
Samba	33.0	39.2	50.5	57.7	23.5	20.2	37.3
GDN-H1	<u>35.6</u>	<u>39.7</u>	<u>52.0</u>	<u>60.1</u>	<u>24.6</u>	<u>22.2</u>	<u>39.0</u>
GDN-H2	38.2	40.4	50.7	63.3	24.8	23.3	40.1

图表解读

表 4 解读：
真实世界检索任务中，Gated DeltaNet 以 30.6% 平均分在递推模型中领先（Mamba2 为 29.8%），但与注意力模型仍有较大差距（Transformer++ 37.0%）。**混合模型 GDN-H2 以 40.1% 超越了所有基线**，证明了混合架构在检索任务中的优势。这也说明检索任务中注意力机制仍是关键组件，纯递推模型在此类任务上有天然短板。

5.3 长度外推

原文翻译

如图 2 所示，我们在六个长文档基准上评估了模型在不同序列长度（4K-20K）下的困惑度。

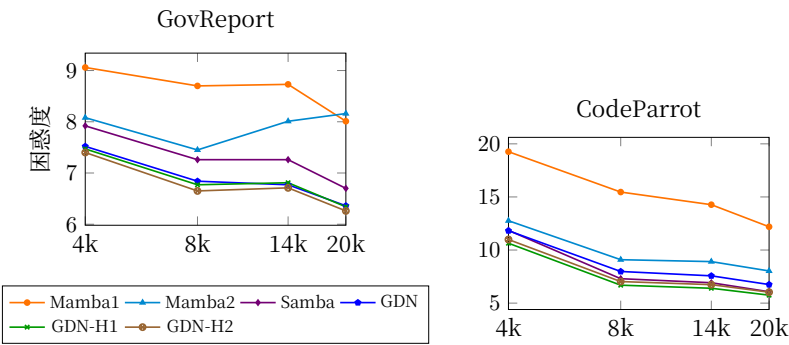


Figure 2: 在两个长文档基准上的长度外推结果（困惑度，越低越好）。训练长度为 4K，评估到 20K。（原论文包含 6 个子图，此处展示 2 个代表性数据集。）

图表解读

图 2 解读：
这组图展示了模型在**超出训练长度**的序列上的表现。所有模型仅在 4K 长度上训练，但在

4K 至 20K 上评估。

关键趋势： Mamba1 的困惑度最高但相对稳定（因为其细粒度门控）； Mamba2 在 Gov-Report 上出现了困惑度回升（长序列退化），而在 CodeParrot 上表现不错； Samba（混合模型）在各基准上表现良好； **Gated DeltaNet 及其混合变体在所有数据集上都保持了最低或接近最低的困惑度**，且随序列增长没有明显退化。

这证明了门控 delta 规则的内存管理能力——它既能保持重要信息（delta 规则），又能及时清除过时信息（门控），因此在长序列上不会出现内存饱和问题。

5.4 长上下文理解

Table 5: LongBench 14 项任务的准确率。NQA=Narrative QA, QQA=QasperQA, MFQ=MultiField QA, HQA=HotpotQA, 2WM=2WikiMulti QA, Mus=Musique, GvR=GovReport, QMS=QMSum, MNs=MultiNews, TRC=TRec, TQA=Trivia QA, SSM=SamSum, LCC=LCC, RBP=RepoBench-P。

Model	单文档QA			多文档QA			摘要			少样本			代码		Avg
	NQA	QQA	MFQ	HQA	2WM	Mus	GvR	QMS	MNs	TRC	TQA	SSM	LCC	RBP	
递推模型															
RetNet	12.1	10.7	19.1	10.7	18.0	5.8	4.8	15.8	7.9	19.0	18.0	<u>12.8</u>	14.1	17.9	13.2
HGRN2	10.7	<u>12.1</u>	19.1	11.3	15.7	<u>6.0</u>	5.2	15.1	9.2	16.0	15.8	10.3	<u>18.6</u>	<u>20.8</u>	13.5
Mamba	<u>13.0</u>	10.1	20.4	10.1	<u>16.7</u>	<u>6.0</u>	<u>7.2</u>	<u>15.9</u>	<u>8.4</u>	<u>23.1</u>	21.9	11.2	17.9	19.0	<u>14.6</u>
DeltaNet	12.9	10.8	<u>21.5</u>	<u>10.9</u>	13.2	5.1	6.5	13.5	7.2	15.5	<u>23.3</u>	11.6	17.6	20.3	13.6
Mamba2	11.1	11.3	18.6	11.8	15.1	6.7	6.7	14.5	7.4	13.0	23.6	8.4	17.9	20.6	13.5
GDN	14.1	14.0	23.3	13.7	14.4	5.8	7.5	16.4	7.9	30.0	22.4	23.0	18.7	22.1	16.6
注意力或混合模型															
Xfmr++	11.8	9.3	10.0	10.9	4.2	6.1	7.4	15.8	6.6	16.9	13.5	3.9	17.2	18.7	11.0
Samba	12.5	<u>12.9</u>	25.4	11.2	19.7	<u>6.8</u>	<u>9.1</u>	15.7	11.0	20.0	<u>22.7</u>	22.8	<u>18.1</u>	<u>21.1</u>	<u>15.9</u>
GDN-H1	14.5	12.3	<u>26.6</u>	<u>12.6</u>	23.6	6.1	<u>9.1</u>	<u>16.1</u>	<u>12.8</u>	<u>33.5</u>	23.9	<u>26.8</u>	15.5	19.2	17.8
GDN-H2	<u>12.7</u>	13.0	<u>27.1</u>	<u>12.7</u>	<u>20.6</u>	7.5	10.4	16.2	13.0	40.5	<u>22.7</u>	27.9	19.9	22.1	18.4

图表解读

表 5 解读：

LongBench 是一个全面的长上下文理解基准。**Gated DeltaNet 在递推模型中以 16.6% 的平均分遥遥领先**（次优的 Mamba 为 14.6%）。特别值得注意的是 TRec（少样本分类）任务中 Gated DeltaNet 达到 30.0%，远超 Mamba2 的 13.0%，以及 SamSum（对话摘要）中的 23.0% vs 8.4%。

混合模型 GDN-H2 以 18.4% 平均分达到最高，且纯 Transformer++ 仅为 11.0%——这再次说明长上下文理解并非 Transformer 的强项（在小模型/小数据设置下），而递推模型和混合模型凭借线性复杂度的推理和有效的内存管理反而表现更好。

5.5 训练吞吐量对比

原文翻译

图 3 展示了不同模型在不同序列长度下的训练吞吐量对比。分析表明，门控 delta 规则相比原始 delta 规则仅引入了边际开销，Gated DeltaNet 的吞吐量与 DeltaNet 基本一致。两者都略慢于 Mamba2（约 2-3K tokens/sec），因为其更具表达力的转移矩阵。Transformer++ 在 2K 窗口下达到最佳性能，得益于高度优化的 Flash-Attention-2 内核。因此，结合 2K 窗口大小 SWA 注意力与其他 token 混合器的混合方法展现了更高的吞吐量。值得注意的是，Gated DeltaNet-H1 在所有序列长度上都保持了令人信服的训练吞吐量。

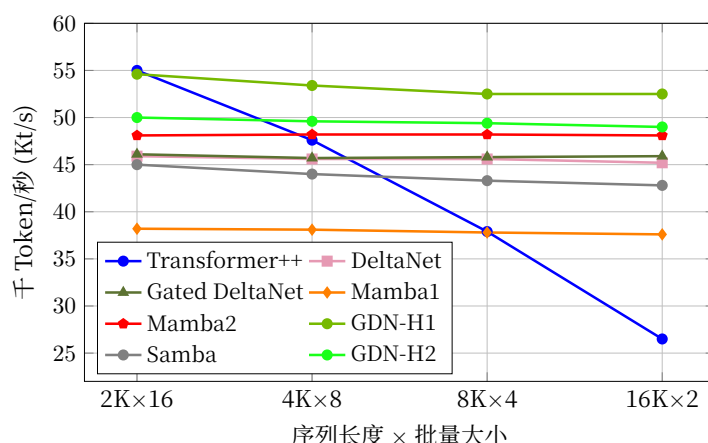


Figure 3: 1.3B 模型在单张 H100 GPU 上的训练吞吐量对比。

图表解读

图 3 解读：

横轴为不同的序列长度×批量大小配置（总 token 数基本恒定），纵轴为训练吞吐量（千 token/秒，越高越好）。

关键观察：

- **Transformer++**（蓝线）在短序列时最快（55K），但随序列增长急剧下降（16K 时仅 26.5K），体现了二次复杂度的代价。
- **Gated DeltaNet**（绿三角）和 **DeltaNet**（紫方块）几乎重合（~46K），证明门控项不增加计算开销。
- **Mamba2**（红五角）略快（~48K），因为对角矩阵计算比 Householder 矩阵简单。
- **GDN-H1**（亮绿）保持 ~52-55K 的高吞吐量，在长序列下远超 Transformer++，这使其成为实际部署的最佳选择。

6 第5节：相关工作 (Related Work)

原文翻译

门控线性 RNN。 大型线性递推语言模型因其训练和推理效率而受到广泛关注。线性 RNN 领域从使用数据无关的衰减机制（如 S4、S5、LRU、RWKV4/5、RetNet）迅速发展到现在在更近期的架构中引入数据依赖的衰减机制（如 HGRN1/2、Mamba1/2、RWKV6、GSA）。这一转变源于门控/遗忘机制（Mamba 中称为选择性机制）的已证优势——这是一个源自门控 RNN 文献的经典概念，其重要性已被多次重新确认。

现代遗忘门与传统设计（如 LSTM 中的）的不同之处在于，它们移除了对前一隐状态的依赖，仅依赖输入数据。这一修改实现了跨序列长度的高效并行化。DeltaNet 缺乏遗忘门一直是其显著的局限性，我们的门控扩展以自然、有效且硬件高效的方式解决了这一差距。

我们还注意到一项近期的并行工作 RWKV-7，使用了类似的想法，但采用了更灵活的对角加低秩转移矩阵形式： $\mathbf{S}_t = \mathbf{S}_{t-1}(\text{diag}(\mathbf{d}_t) - \mathbf{a}_t \mathbf{b}_t^\top) + \mathbf{v}_t \mathbf{k}_t^\top$ 。分块算法可以类似地适应这种情况。

Delta 规则。 delta 学习规则相比 Hebbian 学习具有优越的内存容量，DeltaNet 利用了这一优势，而线性 Transformer 依赖类 Hebbian 规则。Yang et al. (2024) 并行化了 delta 规则计算，并展示了 DeltaNet 的数据依赖单位加低秩结构 $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ 比 Mamba2 的数据依赖对角矩阵 $(\alpha_t \mathbf{I})$ 提供了更大的灵活性。这种结构优势可以实现包括正则语言识别和超越 TC^0 复杂度的状态追踪等复杂推理——这对编码和推理应用至关重要。

尽管有这些显著优势，delta 规则面临理论限制，且在真实世界数据集上仅表现一般，表明仍有改进空间。近期工作提出了一些在不牺牲并行性的情况下提升状态追踪性能的增强方法，包括使用负特征值和多个 Householder 转移矩阵乘积，这些方法可以无缝应用于 Gated DeltaNet。

混合模型。 在本工作中，我们探索了跨层交错混合注意力层的方法，这在 MiniMax-01 和 Hybrid Mamba2-Attention 等模型中广泛使用。在单层内研究混合线性/softmax 注意力也很有趣。

关键概念/总结

相关工作总结： 本文处于线性 RNN/线性 Transformer 研究的前沿，核心创新在于：

1. 首次将门控与 delta 规则结合（虽然 RWKV-7 并行开展了类似工作）。
2. 保持了完全的硬件高效性（与 DeltaNet 同等吞吐量）。
3. delta 规则的 $(\mathbf{I} - \beta \mathbf{k} \mathbf{k}^T)$ 转移矩阵比 Mamba2 的 $\alpha \mathbf{I}$ 更具表达力——理论上可以进行更复杂的计算（超越 TC^0 ）。

7 第6节：结论 (Conclusion)

原文翻译

在本工作中，我们引入了 Gated DeltaNet，它相比 Mamba2 实现了更好的键值关联学习，相比 DeltaNet 实现了更自适应的内存清除，从而在各种任务中持续取得了更好的实证结果。我们从 Yang et al. (2024) 扩展了并行算法，以实现 Gated DeltaNet 的硬件高效训练。我们的混合 Gated DeltaNet 模型实现了更高的训练吞吐量和整体性能，使其非常适合实际部署。

致谢 (Acknowledgment)

原文翻译

感谢 Yu Zhang 协助图表创建和模型评估；感谢 Kazuki Irie 对草稿提供宝贵反馈；感谢 Simeng Sun 和 Zhixuan Lin 对长序列任务评估设置的深入讨论；感谢 Eric Alcaide 和 Volodymyr Kyrlyov 对 DeltaNet 在线学习视角的有益讨论。

A 附录 A：门控 Delta 规则的扩展 WY 表示证明

原文翻译

为减少符号混乱，我们仅考虑第一个块。

对于 $\mathbf{S}_t$ ，扩展 WY 表示为：

$$\mathbf{S}_t = \sum_{i=1}^t \frac{\gamma_t}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^T, \quad \mathbf{u}_t = \beta_t \left(\mathbf{v}_t - \sum_{i=1}^{t-1} \frac{\gamma_t}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^T \mathbf{k}_t \right)$$

我们通过数学归纳法证明。

证明:

$$\begin{aligned}
\mathbf{S}_{t+1} &= \mathbf{S}_t (\alpha_{t+1}(\mathbf{I} - \beta_{t+1} \mathbf{k}_{t+1} \mathbf{k}_{t+1}^\top)) + \beta_{t+1} \mathbf{v}_{t+1} \mathbf{k}_{t+1}^\top \\
&= \alpha_{t+1} \left(\sum_{i=1}^t \frac{\gamma_t}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^\top \right) - \alpha_{t+1} \beta_{t+1} \left(\sum_{i=1}^t \frac{\gamma_t}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^\top \mathbf{k}_{t+1} \mathbf{k}_{t+1}^\top \right) + \beta_{t+1} \mathbf{v}_{t+1} \mathbf{k}_{t+1}^\top \\
&= \sum_{i=1}^t \frac{\gamma_{t+1}}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^\top + \underbrace{\beta_{t+1} \left(\mathbf{v}_{t+1} - \sum_{i=1}^t \frac{\gamma_{t+1}}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^\top \mathbf{k}_{t+1} \right)}_{\mathbf{u}_{t+1}} \mathbf{k}_{t+1}^\top \\
&= \sum_{i=1}^{t+1} \frac{\gamma_{t+1}}{\gamma_i} \mathbf{u}_i \mathbf{k}_i^\top
\end{aligned}$$

其中第二步到第三步利用了 $\alpha_{t+1} \cdot \frac{\gamma_t}{\gamma_i} = \frac{\gamma_{t+1}}{\gamma_i}$ (因为 $\gamma_{t+1} = \alpha_{t+1} \cdot \gamma_t$)，最后一步利用了 $\frac{\gamma_{t+1}}{\gamma_{t+1}} = 1$ 。□

通俗解释

这个证明是 Gated DeltaNet 算法正确性的数学基础。核心思路是：假设在时间步 t 时， $\mathbf{S}_t$ 可以分解为一系列经过衰减的外积之和（WY 形式），然后证明在应用一步门控 delta 规则后， $\mathbf{S}_{t+1}$ 仍然保持这种形式。

关键技巧是衰减因子的“链式传递”： $\alpha_{t+1} \cdot \frac{\gamma_t}{\gamma_i} = \frac{\gamma_{t+1}}{\gamma_i}$ 。这使得所有旧项的衰减因子可以“自动更新”，新项 $\mathbf{u}_{t+1}$ 的衰减因子恰好为 $\frac{\gamma_{t+1}}{\gamma_{t+1}} = 1$ 。

B 附录 B：消融实验

原文翻译

表 6 展示了 Gated DeltaNet 模块组件的消融研究。实验表明短卷积和输出门对模型性能至关重要，而输出归一化的改进较为边际。与 Yang et al. (2024) 一致，L2 归一化对最优性能至关重要，但特征映射的选择影响较小。通过经验分析，头维度 128 提供了性能和计算效率之间的最佳平衡。

表 7 表明在各种混合架构中，Mamba2、Gated DeltaNet、SWA 按此特定顺序的组合产生了最优结果。

Table 6: Gated DeltaNet 模块的消融研究。所有模型为 400M 参数，在 FineWeb-Edu 数据集上训练 15B token。

Gated DeltaNet 消融 (400M)	Avg-PPL (↓)	Avg-Acc (↑)
Gated DeltaNet w Head Dim 128	27.35	47.26
宏观设计		
w. naive Delta Rule	30.87	45.12
w/o. Short Conv	28.95	46.16
w/o. Output Gate	29.12	45.46
w/o. Output Norm	27.55	47.07
归一化与特征映射		
w. L_1 -norm & ReLU	30.79	45.92
w. L_1 -norm & 1+ELU	30.34	46.05
w. L_1 -norm & SiLU	30.18	46.09
w. L_2 -norm & ReLU	27.67	46.94
w. L_2 -norm & 1+ELU	27.58	47.17
模型维度		
w. Head Dim 64	28.31	46.35
w. Head Dim 256	27.13	47.38

图表解读

表 6 解读：
消融实验揭示了几个关键设计选择：

- **最重要：**使用门控 delta 规则（而非朴素 delta 规则，PPL 27.35 vs 30.87），输出门（去掉后 PPL 29.12）。
- **较重要：**短卷积（去掉后 PPL 28.95），L2 归一化（L1 归一化 PPL 30+，L2 归一化 PPL ~27.6）。
- **不太重要：**输出归一化（去掉后 PPL 仅从 27.35 升至 27.55），头维度（128 与 256 差异不大）。

Table 7: 混合架构的消融研究。所有模型为 500M 参数，训练 15B token。

模型	Wiki. ppl↓	LMB. ppl↓	LMB. acc	PIQA acc	Hella. acc_n	Wino. acc	ARC-e acc	ARC-c acc_n	SIQA acc	BoolQ acc	Avg.
GDN + SWA + Mamba2	24.02	28.20	34.77	67.08	40.84	50.74	60.35	28.83	38.94	61.49	47.88
GDN + Mamba2 + SWA	23.69	26.83	36.17	67.51	41.51	51.85	61.19	29.77	38.58	53.73	47.54
Mamba2 + SWA + GDN	24.14	25.21	36.79	64.96	41.18	52.01	60.90	30.03	38.07	59.44	47.92
Mamba2 + GDN + SWA	23.54	24.11	36.92	66.48	41.70	52.72	61.06	30.54	39.91	60.51	48.73

图表解读

表 7 解读：
四种层顺序的对比中，**Mamba2 → Gated DeltaNet → SWA** 以 48.73% 的平均准确率和 23.54/24.11 的困惑度胜出。这一发现表明：

- Mamba2 放在底层负责初步的序列建模和信息筛选。
- Gated DeltaNet 放在中层负责精确的键值关联和记忆管理。
- SWA 放在顶层负责局部精细交互和上下文整合。

这种”从粗到精”的层级组织结构体现了各组件的互补角色。

C 附录 C：参考文献一览表

Table 8: 本文引用的主要参考文献一览

文献	主要内容	在本文中的引用原因
Katharopoulos+ 2020	提出线性 Transformer, 用核化点积替代 softmax	线性注意力的奠基工作
Dao & Gu 2024a (Mamba2)	揭示 SSM 与线性注意力的对偶性 (SSD)	核心基线模型
Dao & Gu 2024b	Mamba2 的 ICML 版本	Mamba2 的正式引用
Gu+ 2023 (Mamba)	提出选择性状态空间模型 Mamba	重要基线模型
Yang+ 2024 (DeltaNet)	并行化 delta 规则计算的 WY 表示算法	Gated DeltaNet 的直接基础
Yang+ 2023 (GLA)	带细粒度衰减的门控线性注意力	分块训练算法的参考
Widrow & Hoff 1988	提出 delta 学习规则	Delta 规则的原始文献
Schlag+ 2021	将 delta 规则应用到线性 Transformer	DeltaNet 的前身工作
Bischof & Loan 1985	WY 表示 (Householder 矩阵乘积的表示法)	并行算法的数学基础
Joffrain+ 2006	UT 变换	WY 表示的矩阵形式推导
Sun+ 2023 (RetNet)	提出 RetNet 架构和保留注意力	基线模型和输出处理的参考
De+ 2024 (Griffin)	RNN + 局部注意力的混合架构	混合模型设计的参考
Ren+ 2024 (Samba)	Mamba + SWA 的混合架构	基线混合模型
Qin+ 2024 (HGRN2)	带层次化门控的递推网络	基线模型
Hsieh+ 2024 (RULER)	提出 NIAH 基准测试套件	S-NIAH 案例研究的测试集
Bai+ 2023 (Long-Bench)	双语多任务长上下文理解基准	长上下文评估基准
Arora+ 2024b	Just Read Twice (JRT)	真实世界检索任务的评估方法
Longhorn (Alizadeh+ 2024)	将 RNN 更新解释为在线学习	Gated DeltaNet 的理论分析框架
TTT (Sun+ 2024)	测试时训练——将隐状态视为可训练参数	SGD/权重衰减视角的参考

(续上表)

文献	主要内容	在本文中的引用原因
Titans (Behrouz+ 2024)	学习在测试时记忆	并行工作中的权重衰减思想
Penedo+ 2024 (FineWeb-Edu)	高质量网络文本训练数据集	所有模型的统一训练数据
Dao 2023 (FlashAttn-2)	Flash Attention 2 的高效实现	SWA 层的高效内核
RWKV-7	并行工作，使用对角+低秩转移矩阵	类似思想的并行工作
Beck+ 2024 (xL-STM)	扩展 LSTM	使用类似衰减机制的模型
Peng+ 2024 (Eagle/RWKV6)	RWKV 第6版	数据依赖衰减的参考
Irie+ (多篇)	快速权重编程系列	delta 规则的理论视角
Grazzi+ 2024	使用负特征值解锁 DeltaNet 状态追踪	delta 规则的扩展方向